This code refers to a `resizeGrid1()` function, shown in the following listing, that's chained to a `resizeGrid2()` function. These functions use the ability to update the size setting on our layout to update the graphical display of the elements created by the layout.

Listing 10.7 The `resizeGrid1()` function

```
function resizeGrid1() {
  grid.size([200,200]);
  grid(doubledArray);

  d3.select("g").selectAll("circle")
    .transition()
    .duration(1000)
    .attr("cx", function(d) {return d.x})
    .attr("cy", function(d) {return d.y})
    .each("end", resizeGrid2);
};

function resizeGrid2() {
  grid.size([200,400]);
  grid(doubledArray);

  d3.select("g").selectAll("circle")
    .transition()
    .duration(1000)
    .attr("cx", function(d) {return d.x;})
    .attr("cy", function(d) {return d.y;});
};
```

Changes the size, reapplies the layout, and updates the display

Again, with a different size

This creates a grid that fits our defined space perfectly, as shown in figure 10.3, and with no need to create a scale to place the elements.

Figure 10.4 shows a pair of animations where the grid changes in size as we adjust the size setting. The grid changes to fit a smaller or an elongated area. This is done using the transition's `"end"` event. It calls a new function that uses our original grid layout but updates its size and reapplies it to our dataset.

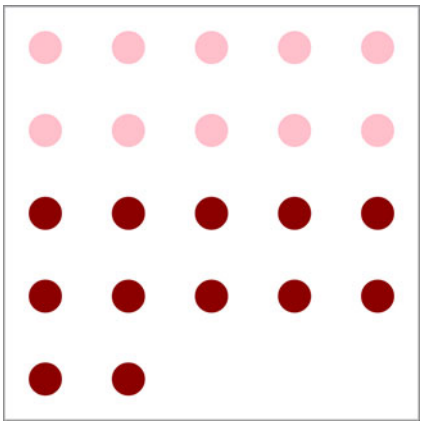


Figure 10.3 The grid layout run on a 400 x 400 size setting